SQL Basics

Assignment Questions

Windows Function:

1

1. Rank the customers based on the total amount they've spent on rentals.

Input

```sql
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(p.amount) DESC) AS spending_rank
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

customer_id	first_name	last_name	total_spent	spending_rank
526	KARL	SEAL	221.55	1
148	ELEANOR	HUNT	216.54	2
144	CLARA	SHAW	195.58	3
137	RHONDA	KENNEDY	194.61	4
178	MARION	SNYDER	194.61	4
459	TOMMY	COLLAZO	186.62	6
469	WESLEY	BULL	177.60	7
468	TIM	CARY	175.61	8
236	MARCIA	DEAN	175.58	9
181	ANA	BRADLEY	174.66	10
176	JUNE	CARROLL	173.63	11
259	LENA	JENSEN	170.67	12
50	DIANE	COLLINS	169.65	13
522	ARNOLD	HAVENS	167.67	14
410	CURTIS	IRBY	167.62	15
403	MIKE	WAY	166.65	16
295	DAISY	BATES	162.62	17
209	TONYA	CHAPMAN	161.68	18

Output

2

2. Calculate the cumulative revenue generated by each film over time.

Input

```sql
SELECT
    f.title,
    p.payment_date,
    SUM(p.amount) OVER (PARTITION BY f.film_id ORDER BY p.payment_date) AS cumulative_revenue
FROM payment p
JOIN rental r ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id;
```

Result Grid | Filter Rows: | Export:

title	payment_date	cumulative_revenue
ACADEMY DINOSAUR	2005-05-27 07:03:28	0.99
ACADEMY DINOSAUR	2005-05-30 20:21:07	2.98
ACADEMY DINOSAUR	2005-06-15 02:57:51	3.97
ACADEMY DINOSAUR	2005-06-17 20:24:00	4.96
ACADEMY DINOSAUR	2005-06-21 00:30:26	6.95
ACADEMY DINOSAUR	2005-07-07 10:41:31	7.94
ACADEMY DINOSAUR	2005-07-07 20:59:06	8.93
ACADEMY DINOSAUR	2005-07-08 19:03:15	9.92
ACADEMY DINOSAUR	2005-07-10 13:07:31	10.91

Output

3

3. Determine the average rental duration for each film, considering films with similar lengths.

Input

```
20 •  SELECT
21        f.film_id,
22        f.title,
23        f.length,
24        AVG(DATEDIFF(r.return_date, r.rental_date)) OVER (PARTITION BY f.length) AS avg_duration_same_length
25    FROM film f
26    JOIN inventory i ON f.film_id = i.film_id
27    JOIN rental r ON i.inventory_id = r.inventory_id;
```

Output

Result Grid | Filter Rows: | Export: | Wrap

film_id	title	length	avg_duration_same_length
469	IRON MOON	46	5.1275
730	RIDGEMONT SUBMARINE	46	5.1275
15	ALIEN CENTER	46	5.1275
730	RIDGEMONT SUBMARINE	46	5.1275
15	ALIEN CENTER	46	5.1275
504	KWAI HOMEWARD	46	5.1275
505	LABYRINTH LEAGUE	46	5.1275
15	ALIEN CENTER	46	5.1275
504	KWAI HOMEWARD	46	5.1275

4

4. Identify the top 3 films in each category based on their rental counts.

Input

```
29 •  SELECT *
30    FROM (
31        SELECT
32            c.name AS category_name,
33            f.title,
34            COUNT(r.rental_id) AS rental_count,
35            RANK() OVER (PARTITION BY c.name ORDER BY COUNT(r.rental_id) DESC) AS rank_per_category
36        FROM category c
37        JOIN film_category fc ON c.category_id = fc.category_id
38        JOIN film f ON f.film_id = fc.film_id
39        JOIN inventory i ON f.film_id = i.film_id
40        JOIN rental r ON i.inventory_id = r.inventory_id
41        GROUP BY c.name, f.title
42    ) AS ranked_films
43    WHERE rank_per_category <= 3;
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

category_name	title	rental_count	rank_per_category
Action	RUGRATS SHAKESPEARE	30	1
Action	SUSPECTS QUILLS	30	1
Action	HANDICAP BOONDOCK	28	3
Action	STORY SIDE	28	3
Action	TRIP NEWTON	28	3
Animation	JUGGLER HARDLY	32	1
Animation	DOGMA FAMILY	30	2
Animation	STORM HAPPINESS	29	3
Children	ROBBERS JOON	31	1
Children	IDOLS SNATCHERS	30	2

5

5. Calculate the difference in rental counts between each customer's total rentals and the average rentals across all customers

Input

```
45 •  SELECT
46        customer_id,
47        total_rentals,
48        total_rentals - AVG(total_rentals) OVER () AS difference_from_avg
49    FROM (
50        SELECT
51            c.customer_id,
52            COUNT(r.rental_id) AS total_rentals
53        FROM customer c
54        JOIN rental r ON c.customer_id = r.customer_id
55        GROUP BY c.customer_id
56    ) AS customer_rentals;
```

Output

Result Grid | Filter Rows: | Exp

customer_id	total_rentals	difference_from_avg
1	32	5.2154
2	27	0.2154
3	26	-0.7846
4	22	-4.7846
5	38	11.2154
6	28	1.2154
7	33	6.2154
8	24	-2.7846
9	23	-3.7846

6

6. Find the monthly revenue trend for the entire rental store over time

Input

```
58 •  SELECT
59        DATE_FORMAT(payment_date, '%Y-%m') AS month,
60        SUM(amount) AS monthly_revenue
61    FROM payment
62    GROUP BY month
63    ORDER BY month;
```

Output

Result Grid | Filter Rows:

month	monthly_revenue
2005-05	4824.43
2005-06	9631.88
2005-07	28373.89
2005-08	24072.13
2006-02	514.18

7

7. Identify the customers whose total spending on rentals falls within the top 20% of all customers.

Input

```sql
SELECT *
FROM (
    SELECT
        c.customer_id,
        c.first_name,
        c.last_name,
        SUM(p.amount) AS total_spent,
        NTILE(5) OVER (ORDER BY SUM(p.amount) DESC) AS quintile
    FROM customer c
    JOIN payment p ON c.customer_id = p.customer_id
    GROUP BY c.customer_id
) AS spending_distribution
WHERE quintile = 1;
```

Result Grid | Filter Rows: | Export:

customer_id	first_name	last_name	total_spent	quintile
526	KARL	SEAL	221.55	1
148	ELEANOR	HUNT	216.54	1
144	CLARA	SHAW	195.58	1
137	RHONDA	KENNEDY	194.61	1
178	MARION	SNYDER	194.61	1
459	TOMMY	COLLAZO	186.62	1
469	WESLEY	BULL	177.60	1
468	TIM	CARY	175.61	1

Output

8

8. Calculate the running total of rentals per category, ordered by rental count.

Input

```
79 •  SELECT
80        c.name AS category_name,
81        COUNT(r.rental_id) AS rental_count,
82        SUM(COUNT(r.rental_id)) OVER (ORDER BY COUNT(r.rental_id)) AS running_total
83    FROM category c
84    JOIN film_category fc ON c.category_id = fc.category_id
85    JOIN film f ON fc.film_id = f.film_id
86    JOIN inventory i ON f.film_id = i.film_id
87    JOIN rental r ON i.inventory_id = r.inventory_id
88    GROUP BY c.name;
```

Result Grid | Filter Rows:

category_name	rental_count	running_total
Music	830	830
Travel	837	1667
Horror	846	2513
Classics	939	3452
New	940	4392
Comedy	941	5333
Children	945	6278
Games	969	7247
Foreign	1033	8280

Output

9

9. Find the films that have been rented less than the average rental count for their respective categories.

Input

```sql
WITH film_rental_counts AS (
    SELECT
        f.film_id,
        f.title,
        c.name AS category_name,
        COUNT(r.rental_id) AS rental_count
    FROM film f
    JOIN film_category fc ON f.film_id = fc.film_id
    JOIN category c ON fc.category_id = c.category_id
    JOIN inventory i ON f.film_id = i.film_id
    JOIN rental r ON i.inventory_id = r.inventory_id
    GROUP BY f.film_id, c.name
)
SELECT *
FROM (
    SELECT *,
           AVG(rental_count) OVER (PARTITION BY category_name) AS avg_rental_in_category
    FROM film_rental_counts
) AS category_avg
WHERE rental_count < avg_rental_in_category;
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

film_id	title	category_name	rental_count	avg_rental_in_category
29	ANTITRUST TOMATOES	Action	10	18.2295
56	BAREFOOT MANCHURIAN	Action	18	18.2295
105	BULL SHAWSHANK	Action	16	18.2295
111	CADDYSHACK JEDI	Action	16	18.2295
126	CASUALTIES ENCINO	Action	9	18.2295
194	CROW GREASE	Action	12	18.2295
205	DANCES NONE	Action	14	18.2295
210	DARKO DORADO	Action	11	18.2295
212	DARN FORRESTER	Action	18	18.2295

10

10. Identify the top 5 months with the highest revenue and display the revenue generated in each month.

Input

```
111 •  SELECT *
112    FROM (
113        SELECT
114            DATE_FORMAT(payment_date, '%Y-%m') AS month,
115            SUM(amount) AS total_revenue,
116            RANK() OVER (ORDER BY SUM(amount) DESC) AS month_rank
117        FROM payment
118        GROUP BY month
119    ) AS ranked_months
120    WHERE month_rank <= 5;
```

Result Grid | Filter Rows:

month	total_revenue	month_r
2005-07	28373.89	1
2005-08	24072.13	2
2005-06	9631.88	3
2005-05	4824.43	4
2006-02	514.18	5

Output

THNKS FOR THIS ASSIGNMENT